Assingment—11

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Prompt: generate a Stack class with push, pop, peek, and is_empty methods.

Code:

```
class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        if not self.is_empty():
            return self.items.pop()
        else:
            raise IndexError("pop from empty stack")

    def peek(self):
        if not self.is_empty():
            return self.items[-1]
        else:
            raise IndexError("peek from empty stack")

    def is_empty(self):
        return len(self.items) == 0
```

```
#example usage
if __name__ == "__main__":
    stack = Stack()
    stack.push(1)
    stack.push(2)
    stack.push(3)
    print(stack.peek())  # Output: 3
    print(stack.pop())   # Output: 3
    print(stack.is_empty())  # Output: False
    print(stack.pop())   # Output: 2
    print(stack.pop())   # Output: 1
    print(stack.is_empty())  # Output: True
```

output:

```
PS C:\Users\prath\.vscode\github> python -u "c:\Users\prath\.vscode\github\tempCodeRunnerFile.py"
3
3
False
2
1
True
PS C:\Users\prath\.vscode\github>
```

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Prompt: generate a python code based on queue class that should execute all topics like enqueue,dequeue,peek and size methods.

Code:

```
class Queue:
    def __init__(self):
```

```
    self.items = []

def enqueue(self, item):
    """Adds an item to the end of the queue."""
    self.items.append(item)
    print(f"Enqueued: {item}")

def dequeue(self):
    """Removes and returns the item from the front of the queue."""
    if not self.is_empty():
        dequeued_item = self.items.pop(0)
        print(f"Dequeued: {dequeued_item}")
        return dequeued_item
    else:
        print("Queue is empty. Cannot dequeue.")
        return None

def peek(self):
    """Returns the item at the front of the queue without removing it."""
    if not self.is_empty():
        item = self.items[0]
        print(f"Peeked: {item}")
        return item
    else:
        print("Queue is empty. No item to peek.")
        return None

def size(self):
```

```
        """Returns the number of items in the queue."""
        current_size = len(self.items)
        print(f"Current queue size: {current_size}")
        return current_size

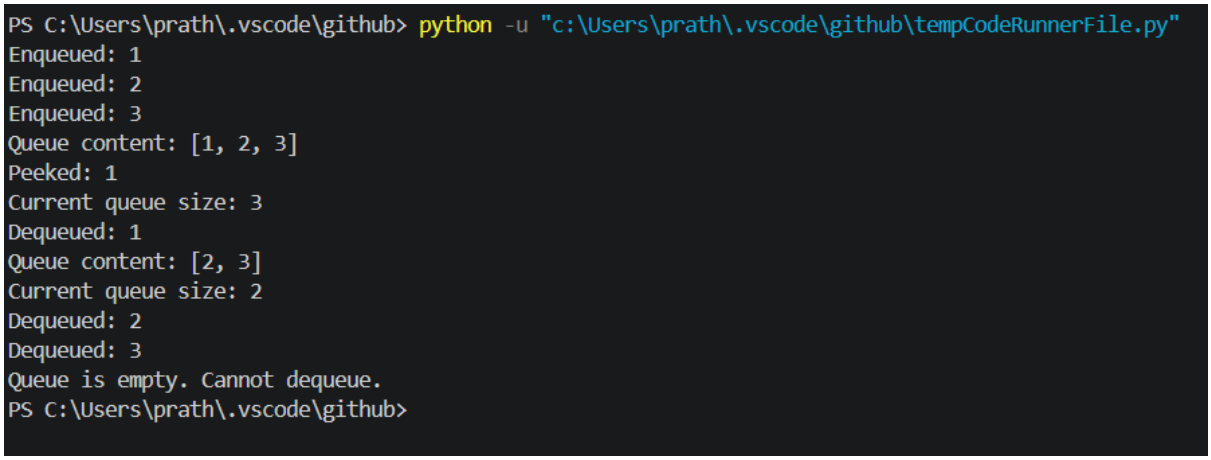
    def is_empty(self):
        """Checks if the queue is empty."""
        return len(self.items) == 0

    def display(self):
        """Displays the current state of the queue."""
        print(f"Queue content: {self.items}")

# Example usage
if __name__ == "__main__":
    queue = Queue()
    queue.enqueue(1)
    queue.enqueue(2)
    queue.enqueue(3)
    queue.display()  # Output: Queue content: [1, 2, 3]
    queue.peek()    # Output: Peeked: 1
    queue.size()    # Output: Current queue size: 3
    queue.dequeue()  # Output: Dequeued: 1
    queue.display()  # Output: Queue content: [2, 3]
    queue.size()    # Output: Current queue size: 2
    queue.dequeue()  # Output: Dequeued: 2
    queue.dequeue()  # Output: Dequeued: 3
    queue.dequeue()  # Output: Queue is empty. Cannot dequeue.
```

```
queue.is_empty() # Output: True
```

output:



Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Prompt: generate a Singly Linked List with insert and display methods.

Code:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def insert_at_beginning(self, data):
        """Inserts a new node at the beginning of the list."""
```

```
    new_node = Node(data)
    new_node.next = self.head
    self.head = new_node
    print(f"Inserted {data} at the beginning.")

def insert_at_end(self, data):
    """Inserts a new node at the end of the list."""
    new_node = Node(data)
    if self.head is None:
        self.head = new_node
        print(f"Inserted {data} at the end (list was empty).")
        return
    current = self.head
    while current.next:
        current = current.next
    current.next = new_node
    print(f"Inserted {data} at the end.")

def display(self):
    """Prints the elements of the linked list."""
    elements = []
    current = self.head
    while current:
        elements.append(current.data)
        current = current.next
    if elements:
        print("Linked List: " + " -> ".join(map(str, elements)))
    else:
```

```
        print("Linked List is empty.")

# Create a new singly linked list
my_sll = SinglyLinkedList()

# Display empty list
my_sll.display()

# Insert elements at the end
my_sll.insert_at_end(10)
my_sll.insert_at_end(20)
my_sll.insert_at_end(30)
my_sll.display()

# Insert elements at the beginning
my_sll.insert_at_beginning(5)
my_sll.insert_at_beginning(1)
my_sll.display()

# Insert another element at the end
my_sll.insert_at_end(40)
my_sll.display()
```

output:

```
PS C:\Users\prath\.vscode\github> python -u "c:\Users\prath\.vscode\github\tempCodeRunnerFile.py"
Linked List is empty.
Inserted 10 at the end (list was empty).
Inserted 20 at the end.
Inserted 30 at the end.
Linked List: 10 -> 20 -> 30
Inserted 5 at the beginning.
Inserted 1 at the beginning.
Linked List: 1 -> 5 -> 10 -> 20 -> 30
Inserted 40 at the end.
Linked List: 1 -> 5 -> 10 -> 20 -> 30 -> 40
PS C:\Users\prath\.vscode\github>
```

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Prompt: create a python code to get a BST with insert and in-order traversal methods.

Code:

```
class BSTNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

class BST:
    def __init__(self):
        self.root = None

    def insert(self, data):
        if self.root is None:
            self.root = BSTNode(data)
        else:
            self._insert_recursive(self.root, data)
```

```
    def _insert_recursive(self, node, data):
        if data < node.data:
            if node.left is None:
                node.left = BSTNode(data)
            else:
                self._insert_recursive(node.left, data)
        else:
            if node.right is None:
                node.right = BSTNode(data)
            else:
                self._insert_recursive(node.right, data)

    def in_order_traversal(self):
        result = []
        self._in_order_recursive(self.root, result)
        return result

    def _in_order_recursive(self, node, result):
        if node:
            self._in_order_recursive(node.left, result)
            result.append(node.data)
            self._in_order_recursive(node.right, result)
#example usage
if __name__ == "__main__":
    bst = BST()
    bst.insert(50)
    bst.insert(30)
```

```
bst.insert(70)
bst.insert(20)
bst.insert(40)
bst.insert(60)
bst.insert(80)

print("In-order Traversal of the BST:", bst.in_order_traversal())
```

output:

```
PS C:\Users\prath\.vscode\github> python -u "c:\Users\prath\.vscode\github\tempCodeRunnerFile.py"
In-order Traversal of the BST: [20, 30, 40, 50, 60, 70, 80]
PS C:\Users\prath\.vscode\github>
```

Task Description #5 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq module.

Prompt: generate a python code to implement a priority queue using Python's heapq module.

Code:

```
import heapq


from matplotlib import collections


class PriorityQueue:
    def __init__(self):
        self._queue = []
        self._index = 0

    def push(self, item, priority):
```

```
    """Adds an item to the priority queue with a given priority.
    The priority is typically a number, with lower numbers indicating higher priority.
    """
    # We use a tuple (-priority, index, item) to handle:
    # 1. Min-heap behavior for max-priority (hence -priority)
    # 2. Stable ordering for items with the same priority (using index)
    # 3. Storing the actual item
    heapq.heappush(self._queue, (-priority, self._index, item))
    print(f"Pushed: {item} with priority {priority}")
    self._index += 1

def pop(self):
    """Removes and returns the item with the highest priority.
    Returns None if the queue is empty.
    """
    if self.is_empty():
        print("Priority Queue is empty. Cannot pop.")
        return None
    priority, index, item = heapq.heappop(self._queue)
    print(f"Popped: {item} (original priority {-priority})")
    return item

def peek(self):
    """Returns the item with the highest priority without removing it.
    Returns None if the queue is empty.
    """
    if self.is_empty():
        print("Priority Queue is empty. No item to peek.")
```

```
            return None
        priority, index, item = self._queue[0]
        print(f"Peeked: {item} (original priority {-priority})")
        return item

    def is_empty(self):
        """Checks if the priority queue is empty."""
        return len(self._queue) == 0

    def size(self):
        """Returns the number of items in the priority queue."""
        current_size = len(self._queue)
        print(f"Current priority queue size: {current_size}")
        return current_size

    def display(self):
        """Displays the current state of the priority queue (items and their effective priorities)."""
        displayed_items = []
        for p, idx, item in self._queue:
            displayed_items.append(f"(Item: {item}, Priority: {-p})")
        print(f"Priority Queue content: [{', '.join(displayed_items)}]")
#example usage
if __name__ == "__main__":
    pq = PriorityQueue()
    pq.push("task1", priority=3)
    pq.push("task2", priority=1)
    pq.push("task3", priority=2)
```

pq.display() # Output: Priority Queue content: [(Item: task2, Priority: 1), (Item: task1, Priority: 3), (Item: task3, Priority: 2)]

pq.peek() # Output: Peeked: task2 (original priority 1)

pq.size() # Output: Current priority queue size: 3

pq.pop() # Output: Popped: task2 (original priority 1)

pq.display() # Output: Priority Queue content: [(Item: task3, Priority: 2), (Item: task1, Priority: 3)]

pq.pop() # Output: Popped: task3 (original priority 2)

pq.pop() # Output: Popped: task1 (original priority 3)

pq.pop() # Output: Priority Queue is empty. Cannot pop.

Output:

```
PS C:\Users\prath\.vscode\github> python -u "c:\Users\prath\.vscode\github\tempCodeRunnerFile.py"
Pushed: task1 with priority 3
Pushed: task2 with priority 1
Pushed: task3 with priority 2
Priority Queue content: [(Item: task1, Priority: 3), (Item: task2, Priority: 1), (Item: task3, Priority: 2)]
Peeked: task1 (original priority 3)
Current priority queue size: 3
Popped: task1 (original priority 3)
Priority Queue content: [(Item: task3, Priority: 2), (Item: task2, Priority: 1)]
Popped: task3 (original priority 2)
Popped: task2 (original priority 1)
Priority Queue is empty. Cannot pop.
PS C:\Users\prath\.vscode\github>
```

Task Description #6 – Deque

Task: Use AI to implement a double-ended queue using collections.deque.

Prompt: generate python codeto implement a double-ended queue using collections.deque.

Code:

```
import collections


class DequeWrapper:
```

```
def __init__(self):
    self.dq = collections.deque()
    print("Initialized an empty double-ended queue.")

def append_right(self, item):
    """Adds an item to the right end of the deque."""
    self.dq.append(item)
    print(f"Appended {item} to the right.")

def append_left(self, item):
    """Adds an item to the left end of the deque."""
    self.dq.appendleft(item)
    print(f"Appended {item} to the left.")

def pop_right(self):
    """Removes and returns an item from the right end of the deque."""
    if not self.is_empty():
        item = self.dq.pop()
        print(f"Popped {item} from the right.")
        return item
    else:
        print("Deque is empty. Cannot pop from right.")
        return None

def pop_left(self):
    """Removes and returns an item from the left end of the deque."""
    if not self.is_empty():
        item = self.dq.popleft()
```

```
            print(f"Popped {item} from the left.")
            return item
        else:
            print("Deque is empty. Cannot pop from left.")
            return None

    def peek_right(self):
        """Returns the item at the right end without removing it."""
        if not self.is_empty():
            item = self.dq[-1]
            print(f"Peeked right: {item}")
            return item
        else:
            print("Deque is empty. No item to peek.")
            return None

    def peek_left(self):
        """Returns the item at the left end without removing it."""
        if not self.is_empty():
            item = self.dq[0]
            print(f"Peeked left: {item}")
            return item
        else:
            print("Deque is empty. No item to peek.")
            return None

    def size(self):
        """Returns the number of items in the deque."""
```

```
        current_size = len(self.dq)
        print(f"Current deque size: {current_size}")
        return current_size

    def is_empty(self):
        """Checks if the deque is empty."""
        return len(self.dq) == 0

    def display(self):
        """Displays the current state of the deque."""
        print(f"Deque content: {list(self.dq)}")

#example usage
if __name__ == "__main__":
    deque_wrapper = DequeWrapper()
    deque_wrapper.append_right(1)
    deque_wrapper.append_right(2)
    deque_wrapper.append_left(0)
    deque_wrapper.display()  # Output: Deque content: [0, 1, 2]

    deque_wrapper.peek_left()   # Output: Peeked left: 0
    deque_wrapper.peek_right()  # Output: Peeked right: 2
    deque_wrapper.size()       # Output: Current deque size: 3

    deque_wrapper.pop_left()    # Output: Popped 0 from the left.
    deque_wrapper.pop_right()   # Output: Popped 2 from the right.
    deque_wrapper.display()     # Output: Deque content: [1]
```

```
deque_wrapper.pop_left()    # Output: Popped 1 from the left.
deque_wrapper.pop_left()    # Output: Deque is empty. Cannot pop from left.
```

Output:

```
PS C:\Users\prath\.vscode\github> python -u "c:\Users\prath\.vscode\github\tempCodeRunnerFile.py"
Initialized an empty double-ended queue.
Appended 1 to the right.
Appended 2 to the right.
Appended 0 to the left.
Deque content: [0, 1, 2]
Peeked left: 0
Peeked right: 2
Current deque size: 3
Popped 0 from the left.
Popped 2 from the right.
Deque content: [1]
Popped 1 from the left.
Deque is empty. Cannot pop from left.
PS C:\Users\prath\.vscode\github>
```

Task Description #7 Real-Time Application Challenge – Choose the Right Data Structure

Task: real time application challenge

Prompt: generate python code to implement as real time application challenge to verify the data structures of stack ,queue ,linked list, etc...

Your college wants to develop a Campus Resource Management System

that handles:

1. Student Attendance Tracking – Daily log of students

entering/exiting the campus.

2. Event Registration System – Manage participants in events with

quick search and removal.

3. Library Book Borrowing – Keep track of available books and their

due dates.

4. Bus Scheduling System – Maintain bus routes and stop

connections.

5. Cafeteria Order Queue – Serve students in the order they arrive.

Code:

```
from collections import deque


class CafeteriaQueue:
    """Manages a queue of students for cafeteria orders using a Deque."""

    def __init__(self):
        """Initializes an empty queue."""
        self.queue = deque()
        print("Cafeteria queue system initialized.")

    def add_student(self, student_name):
        """Adds a student to the end of the queue.

        Args:
            student_name (str): The name of the student to add.
        """
        self.queue.append(student_name)
        print(f"{student_name} has been added to the queue.")

    def serve_student(self):
        """Removes and returns the student at the front of the queue.

        Returns:
            str: The name of the student being served, or None if the queue is empty.
        """
        if not self.is_empty():
            served_student = self.queue.popleft()
```

```
        print(f"{served_student} has been served.")
        return served_student
    else:
        print("The queue is empty. No students to serve.")
        return None

def peek_next_student(self):
    """Returns the student at the front of the queue without removing them.

    Returns:
        str: The name of the next student to be served, or None if the queue is empty.
    """
    if not self.is_empty():
        print(f"Next student to be served: {self.queue[0]}")
        return self.queue[0]
    else:
        print("The queue is empty. No next student.")
        return None

def get_queue_size(self):
    """Returns the current number of students in the queue.

    Returns:
        int: The number of students in the queue.
    """
    size = len(self.queue)
    print(f"Current queue size: {size}")
    return size
```

```
    def is_empty(self):
        """Checks if the queue is empty.

        Returns:
            bool: True if the queue is empty, False otherwise.
        """
        return len(self.queue) == 0

    def display_queue(self):
        """Displays all students currently in the queue.
        """
        if not self.is_empty():
            print("Current queue:")
            for i, student in enumerate(self.queue):
                print(f"  {i+1}. {student}")
        else:
            print("The queue is currently empty.")

# --- Demo of Cafeteria Order Queue ---
# Create a new cafeteria queue
cafeteria = CafeteriaQueue()

# Add some students to the queue
cafeteria.add_student("Alice")
cafeteria.add_student("Bob")
cafeteria.add_student("Charlie")
```

```
# Display the current queue
cafeteria.display_queue()

# Check the next student to be served
cafeteria.peek_next_student()

# Serve a student
cafeteria.serve_student()

# Display the queue again
cafeteria.display_queue()

# Add another student
cafeteria.add_student("David")

# Serve all remaining students
while not cafeteria.is_empty():
    cafeteria.serve_student()

# Try to serve from an empty queue
cafeteria.serve_student()

# Display the empty queue
cafeteria.display_queue()
```

Output:

```
PS C:\Users\prath\.vscode\github> python -u "c:\Users\prath\.vscode\github\tempCodeRunnerFile.py"
Cafeteria queue system initialized.
Alice has been added to the queue.
Bob has been added to the queue.
Charlie has been added to the queue.
Current queue:
  1. Alice
  2. Bob
  3. Charlie
Next student to be served: Alice
Alice has been served.
Current queue:
  1. Bob
  2. Charlie
David has been added to the queue.
Bob has been served.
Charlie has been served.
David has been served.
The queue is empty. No students to serve.
The queue is currently empty.
PS C:\Users\prath\.vscode\github>
```

Task Description #8: Smart E-Commerce Platform – Data Structure Challenge

Prompt: generate python code on e-commerce like building an ordering system by using data strucuters like, queue, linked list, etc...

An e-commerce company wants to build a Smart Online Shopping System

with:

1. Shopping Cart Management – Add and remove products

dynamically.

2. Order Processing System – Orders processed in the order they are

placed.

3. Top-Selling Products Tracker – Products ranked by sales count.

4. Product Search Engine – Fast lookup of products using product ID.

5. Delivery Route Planning – Connect warehouses and delivery

locations.

Code:

from collections import deque

```
class OrderProcessingSystem:
    """Manages incoming orders using a Queue to ensure FIFO processing."""

    def __init__(self):
        """Initializes an empty order queue."""
        self.order_queue = deque()
        print("Order Processing System initialized. Ready to process orders.")

    def place_order(self, order_id, order_details):
        """Adds a new order to the end of the processing queue.

        Args:
            order_id (str or int): A unique identifier for the order.
            order_details (dict): A dictionary containing details about the order (e.g., items, customer_id).
        """
        order = {'order_id': order_id, 'details': order_details}
        self.order_queue.append(order)
        print(f"Order {order_id} has been placed and added to the queue.")

    def process_next_order(self):
        """Processes (removes) the oldest order from the front of the queue.

        Returns:
            dict: The processed order, or None if the queue is empty.
        """
        if not self.is_empty():
```

```
        processed_order = self.order_queue.popleft()
        print(f"Processing order {processed_order['order_id']}...")
        # In a real system, further logic would go here (e.g., update inventory, send confirmation)
        return processed_order
    else:
        print("No orders in the queue to process.")
        return None

def peek_next_order(self):
    """Returns the next order to be processed without removing it.

    Returns:
        dict: The next order, or None if the queue is empty.
    """
    if not self.is_empty():
        next_order = self.order_queue[0]
        print(f"Next order to be processed: {next_order['order_id']}")
        return next_order
    else:
        print("No orders in the queue.")
        return None

def get_queue_size(self):
    """Returns the current number of orders waiting to be processed.

    Returns:
        int: The number of orders in the queue.
```

```
        """
        size = len(self.order_queue)
        print(f"Current number of orders in queue: {size}")
        return size

    def is_empty(self):
        """Checks if the order queue is empty.

        Returns:
            bool: True if the queue is empty, False otherwise.
        """
        return len(self.order_queue) == 0

    def display_queue(self):
        """Displays all orders currently waiting in the queue."""
        if not self.is_empty():
            print("--- Current Order Queue (Oldest to Newest) ---")
            for i, order in enumerate(self.order_queue):
                print(f"  {i+1}. Order ID: {order['order_id']}, Details: {order['details']}")
            print("-----------------------------------------------")
        else:
            print("The order queue is currently empty.")

# --- Demo of Order Processing System ---
# Initialize the order processing system
ops = OrderProcessingSystem()

print("\n--- Placing Orders ---")
```

```
# Place a few orders
ops.place_order(101, {'items': ['Laptop', 'Mouse'], 'customer': 'Alice'})
ops.place_order(102, {'items': ['Keyboard'], 'customer': 'Bob'})
ops.place_order(103, {'items': ['Monitor', 'Webcam'], 'customer': 'Charlie'})

print("\n--- Current Queue State ---")
ops.display_queue()
ops.get_queue_size()
ops.peek_next_order()

print("\n--- Processing Orders ---")
# Process the first order
processed_order_1 = ops.process_next_order()
print(f"Successfully processed: {processed_order_1['order_id']}")

print("\n--- Queue State After First Processing ---")
ops.display_queue()
ops.get_queue_size()
ops.peek_next_order()

# Place another order while others are being processed
print("\n--- Placing another order ---")
ops.place_order(104, {'items': ['Headphones'], 'customer': 'David'})

print("\n--- Final Queue Processing ---")
# Process all remaining orders
while not ops.is_empty():
    ops.process_next_order()
```

```
# Try to process from an empty queue

print("\n--- Attempt to process from empty queue ---")

ops.process_next_order()

ops.display_queue()
```

Output:

```
PS C:\Users\prath\.vscode\github> python -u "c:\Users\prath\.vscode\github\tempCodeRunnerFile.py"
Order Processing System initialized. Ready to process orders.

--- Placing Orders ---
Order 101 has been placed and added to the queue.
Order 102 has been placed and added to the queue.
Order 103 has been placed and added to the queue.

--- Current Queue State ---
--- Current Order Queue (Oldest to Newest) ---
  1. Order ID: 101, Details: {'items': ['Laptop', 'Mouse'], 'customer': 'Alice'}
  2. Order ID: 102, Details: {'items': ['Keyboard'], 'customer': 'Bob'}
  3. Order ID: 103, Details: {'items': ['Monitor', 'Webcam'], 'customer': 'Charlie'}
------------------------------------------------
Current number of orders in queue: 3
Next order to be processed: 101

--- Processing Orders ---
Processing order 101...
Successfully processed: 101

--- Queue State After First Processing ---
--- Current Order Queue (Oldest to Newest) ---
  1. Order ID: 102, Details: {'items': ['Keyboard'], 'customer': 'Bob'}
  2. Order ID: 103, Details: {'items': ['Monitor', 'Webcam'], 'customer': 'Charlie'}
------------------------------------------------
Current number of orders in queue: 2
Next order to be processed: 102

--- Placing another order ---
Order 104 has been placed and added to the queue.

--- Final Queue Processing ---
Processing order 102...
Processing order 103...
Processing order 104...

--- Attempt to process from empty queue ---
No orders in the queue to process.
```

